PRÁCTICA DE LABORATORIO 3

OBJETIVO

Familiarizar al estudiante con la sintaxis del lenguaje ensamblador MIPS32 y su ejecución en el entorno MARS, resolviendo problemas clásicos de programación, prestando especial atención a operaciones de **entrada/salida (E/S)** mediante el manejo de dispositivos de hardware mapeados en memoria.

EJERCICIOS

1. Un computador de 32 bits con procesador MIPS32 y mapa de E/S simulada en memoria dispone de un sensor de luminosidad con tres registros de 32 bits definidos en la memoria de datos:
   Registros del sensor
   LuzControl. Escribir 0x1 → inicializa el sensor.
   LuzEstado. 0 → sensor no listo. 1 → lectura disponible. -1 → error de hardware.
   LuzDatos. Contiene la lectura de luminosidad (valor entero entre 0 y 1023).
   Escribir en MIPS32 los siguientes procedimientos:
   InicializarSensorLuz: Inicializa el sensor de luminosidad y espera a que esté listo.
   LeerLuminosidad: Devuelve el valor leído. Devuelve un código de estado: 0 lectura correcta, -1 error.
2. Un módulo para medir la tensión arterial de una persona está conectado a un computador de 32 bits con procesador **MIPS32**. El dispositivo tiene los siguientes registros, definidos en la memoria de datos: **TensionControl, TensionEstado, TensionSistol y TensionDiastol.** El funcionamiento es:

   Escribir el valor 1 en TensionControl para iniciar una medición.
   Mientras TensionEstado sea 0, el dispositivo está midiendo. Cuando vale 1, los resultados están listos.
   Los resultados de la medición (valores enteros de 32 bits) quedan disponibles en TensionSistol y TensionDiastol.

Escribir un procedimiento en MIPS32 etiquetado como controlador_tension que: Inicie la medición. Espere a que TensionEstado tenga el valor de 1. Retorne como salida (en $v0 y $v1) los valores de tensión sistólica y diastólica, respectivamente.

INFORME

1. Explique cómo se organiza la memoria cuando un sistema utiliza **memory-mapped I/O**. ¿En qué región de memoria se suelen mapear los dispositivos? ¿Qué implicaciones tiene para las instrucciones lw y sw?

2. ¿Cuál es la principal diferencia entre **memory-mapped I/O** y la **entrada/salida por puertos**? ¿Qué ventajas y desventajas tiene cada enfoque? ¿Por qué MIPS32 utiliza principalmente memory-mapped I/O?
3. En un sistema con memory-mapped I/O: ¿Qué problemas pueden surgir si dos dispositivos usan direcciones solapadas? ¿Cómo se evita este conflicto?
4. ¿Por qué se considera que el memory-mapped I/O simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos?
5. ¿Qué ocurre a nivel del **bus de datos y direcciones** cuando el procesador accede a una dirección de memoria que corresponde a un dispositivo? ¿Cómo sabe el hardware que debe acceder a un periférico en lugar de la RAM?
6. ¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria? ¿Qué mecanismos de protección existen para evitar accesos no autorizados?
7. ¿Qué técnicas se pueden emplear para evitar esperas activas innecesarias al interactuar con dispositivos?
8. Análisis y Discusión de los Resultados

ENTREGA

1. Práctica de Laboratorio **Individual o en Pareja.**
2. Se deben enviar al correo los archivos .asm junto con el documento (.tex y .pdf) con las respuestas así como enlace al repositorio en Github y recursos adicionales.
3. Se debe garantizar la correcta ejecución de los archivos .asm y visualización del documento (.tex y .pdf) con las respuestas.
4. La defensa de cada práctica de laboratorio con su informe es obligatoria.